

Technical Design Document

Zombie Survival — BMI4242 Game Programming Term Project

Game Title Zombie Survival
Course BMI4242 Game Programming — Spring 2025/2026
Instructor Asst. Prof. Dr. Alpay DORUK
Framework Python 3 + Pygame 2
Repository <https://github.com/tunayamann/zombie-survival>

Team Members & Primary Roles

Name	Student No	Role
Hasbi Tuna Yaman	2211504073	Gameplay & Systems Programmer
Fatih Taşkın	2111504031	AI & Pathfinding Programmer
Melisa Önal	2111504026	UI/UX & Audio Integrator

1. Game Overview

Elevator Pitch: A top-down 2D zombie survival shooter in which the player must survive eight increasingly difficult waves of enemies using precise mouse-aimed shooting and strategic use of dynamically generated cover.

Core Mechanics: - **Movement:** WASD keys with diagonal-speed normalisation for consistent feel in all directions. - **Aiming & Shooting:** The player aims with the mouse cursor; holding the left mouse button fires bullets automatically at a fixed cooldown rate. - **Enemy Waves:** Zombies spawn from the arena edges and navigate toward the player using A* pathfinding. Each wave increases enemy count and movement speed. - **Dynamic Maps:** The tile map is procedurally regenerated at the start of every wave, forcing the player to adapt their strategy each round. - **Name Entry:** The player enters their name before the game begins; it is displayed persistently on the HUD.

Win Condition: Successfully eliminate all enemies across all 8 waves.

Loss Condition: Player health reaches zero.

2. Software Architecture

2.1 High-Level Structure

The project follows a traditional **Object-Oriented Programming (OOP)** hierarchy. Each game entity — Player, Enemy, Bullet, Particle — is a self-contained class responsible for its own state, update logic, and rendering. A central Game class orchestrates all subsystems via a **Finite-State Machine**.

2.2 Core Managers & Controllers

Module	Class	Responsibility
game.py	Game	Main loop, FSM, system orchestration, collision resolution
wave_manager.py	WaveManager	Wave progression, drip-spawn timing, win detection
wave_manager.py	EnemyPool	Pre-allocated enemy object pool
bullet.py	BulletPool	Pre-allocated bullet object pool
collision.py	SpatialGrid	Broad-phase spatial hash for collision detection
pathfinding.py	—	A* algorithm implementation
tile_map.py	—	Map generation and coordinate conversion utilities
sound_manager.py	SoundManager	Programmatic PCM audio synthesis
particles.py	Particle	Visual particle effects for death bursts
ui.py	UI	All non-world rendering (HUD, menus, overlays)
player.py	Player	Player movement, shooting, damage
enemy.py	Enemy	Zombie AI, pathfinding, separation steering, animation

Inter-system communication is deliberately kept one-directional: Game calls into subsystems each frame; subsystems do not hold references to each other. `Enemy.update()` receives `grid` and `player` as arguments rather than storing them, which avoids hidden coupling and makes each class independently testable.

2.3 Design Patterns

Object Pool Pattern — Both `BulletPool` and `EnemyPool` pre-allocate fixed arrays of inactive instances at startup. When an object is needed, the pool returns the first inactive slot and reconfigures it in-place. This eliminates per-frame heap allocation and prevents garbage-collection pauses during gameplay — critical for maintaining a stable 60 FPS.

Finite-State Machine — Game drives all state transitions through six explicit states: MENU → NAME_INPUT → PLAYING ↔ PAUSED, PLAYING → GAME_OVER, PLAYING → WIN. Each state has its own event-handling, update, and draw branch, preventing logic bleed between states.

Separation of Concerns — Each source file has exactly one responsibility. Constants live exclusively in `settings.py`; coordinate math lives in `tile_map.py`; rendering logic lives in `ui.py`. This makes the codebase navigable and each module independently maintainable.

3. Algorithmic Implementation & Data Structures

This is the most technically rich area of the project.

3.1 Data Structures

Tile Grid — 2D Array The game world is represented as a `GRID_ROWS × GRID_COLS` 2D Python list (`list[list[int]]`). Each cell stores 0 (walkable) or 1 (wall). Cell access is $O(1)$ by (row, col) index. The grid is regenerated fresh each wave by the procedural map generator.

A* Open Set — Min-Heap Python's `heapq` module implements the A* open set as a binary min-heap keyed by $f = g + h$. Each push and pop is $O(\log n)$ where n is the number of cells in the open set. A tie-breaking counter is included in the heap tuple to prevent comparison errors between nodes with equal f-scores.

G-Score Table — Hash Dictionary `g_score` is stored as a Python dict mapping (row, col) tuples to float costs. Dictionary lookup is $O(1)$ average, making per-node cost comparisons fast.

Spatial Hash Grid — Uniform Cell Dictionary `SpatialGrid` maps integer (cell_x, cell_y) keys to lists of entities that fall within that 64×64 pixel cell. Inserting an entity is $O(1)$; querying the 3×3 neighbourhood of a point retrieves all candidates in $O(k)$ where k is the number of nearby entities (typically very small).

3.2 Algorithms

A Pathfinding (`pathfinding.py`)*

Each Enemy maintains its own A* path recalculated every 0.45 seconds. Refreshing every frame for up to 17 enemies on a 25×17 grid would cost approximately 7,000+ node expansions per second — unacceptable at 60 FPS. The 0.45-second timer reduces this overhead by ~97% while keeping enemy movement responsive.

Key implementation decisions: - **8-directional movement** allows enemies to take diagonal shortcuts, producing more natural pursuit paths. - **Euclidean distance heuristic** is admissible for 8-directional grids (never overestimates), guaranteeing optimal paths. - **Corner-cut prevention:** A diagonal move (dr, dc) is rejected if either adjacent cardinal cell (r, dc) or (dr, c) is a wall. This prevents enemies from visually clipping through wall corners. - The returned path

excludes the start node and contains world-space (x, y) waypoints converted via `grid_to_world()`.

Separation Steering (`enemy.py`)

A known limitation of pure A* pathfinding is that all enemies converge on the same optimal path, collapsing into a single cluster that is trivially easy to eliminate. To counter this, a **Boids-style separation force** is applied after path-following each frame.

For each active enemy, a repulsion vector is computed by summing unit direction vectors away from all neighbours within a 56-pixel influence radius, weighted by proximity (stronger force when closer). The resultant vector is normalised and applied at 60% of the enemy's base speed. If the candidate new position lies on a wall tile, the move is rejected to prevent enemies from being pushed through geometry.

The two-layer architecture — A* for global navigation, separation for local avoidance — produces emergent flanking behaviour: enemies arrive at the player from multiple directions simultaneously rather than queueing in a single file.

Procedural Map Generation (`tile_map.py`)

A new map is generated at the start of each wave using a **constraint-based random rectangle placement** algorithm:

1. Initialise grid with walkable tiles; stamp a solid 1-tile border.
2. Build a **protected set** of tiles that must remain walkable:
 - A 5×5 zone centred on the map centre (player respawn point).
 - The second row and column from each border edge (the band where WaveManager samples enemy spawn positions).
3. Attempt to place 5–7 + wave // 3 rectangular obstacle clusters at random positions and sizes.
4. Any candidate rectangle that overlaps the protected set is discarded without modifying the grid.

No connectivity verification is performed; the protected spawn band guarantees that enemy start positions are always reachable, and A* handles any unreachable interior regions gracefully by returning an empty path.

Two-Phase Collision Detection (`game.py`, `collision.py`)

Naïve bullet-enemy collision testing is $O(b \times e)$ per frame — with 60 bullets and 17 enemies that is 1,020 distance calculations every 16 ms. The pipeline is reduced to near $O(b)$ using two phases:

1. **Broad phase:** All active enemies are inserted into SpatialGrid (64-pixel cells). For each bullet, only enemies in the bullet's cell and its 8 adjacent cells are retrieved as candidates — typically 0–3 enemies.

2. **Narrow phase:** An exact **circle-circle distance test** (`math.hypot`) is performed only on candidates. A bullet is deactivated on its first confirmed hit.

3.3 Programmatic Audio (`sound_manager.py`)

Rather than bundling external audio files, `SoundManager` synthesises all six sound effects at runtime as **16-bit mono PCM** data. Each tone is a sine wave with a linear decay envelope, packed into an in-memory RIFF/WAVE buffer and loaded directly by `pygame.mixer`. This approach keeps the project self-contained with zero external assets.

4. Optimization & Memory Management

4.1 Performance Bottlenecks & Solutions

Bottleneck	Naive Cost	Optimization	Result
A* per enemy per frame	$O(n \log n) \times 17/\text{frame}$	Timer-based refresh every 0.45 s	~97% reduction in A* calls
Bullet-enemy collision	$O(b \times e) = \sim 1,020/\text{frame}$	Spatial hash broad phase	Near $O(b)$ average case
Separation steering	$O(e^2)$ scan	Pre-built active-enemy list passed to each enemy	Single list allocation per frame
Particle rendering	—	List pruned in-place each frame	No unbounded growth

4.2 Memory Management — Object Pooling

Two object pools are active for the entire game session:

- **BulletPool:** 60 `Bullet` instances. At a fire rate of one bullet per 0.18 seconds and a max bullet lifetime of 2.2 seconds, at most ~12 bullets can be alive simultaneously — well within the pool capacity.
- **EnemyPool:** 60 `Enemy` instances. Maximum enemies alive at any time is 17 (the wave cap), leaving ample headroom.

When an entity is “destroyed” (bullet hits wall, enemy health reaches zero), `deactivate()` resets its active flag. The slot is immediately available for reuse on the next spawn request, with no heap allocation or deallocation occurring during gameplay.

5. Version Control & Workflow

Repository: <https://github.com/tunayamann/zombie-survival>

5.1 Git Strategy

The team used a **Feature Branch Workflow**. The main branch was kept in a consistently playable state. Individual features — pathfinding integration, UI/audio systems, wave management, and procedural generation — were developed on dedicated branches and merged into main after manual testing.

5.2 Integration Challenges

The primary technical integration challenge was the **dual coordinate system**: the rendering layer operates in pixel-space (x, y) while A* operates in grid-space (row, col). An off-by-one error caused by the 40-pixel HUD offset initially produced enemies pathfinding to incorrect tile positions. This was resolved by centralising all coordinate conversion in `tile_map.py` through the `world_to_grid()` and `grid_to_world()` functions — every other module uses these two functions exclusively and never performs raw coordinate arithmetic.

A secondary challenge arose when the separation-steering system caused enemies to be pushed into wall tiles when the horde was densely packed. The fix was to validate the candidate post-separation position against the grid before applying the move, rejecting any update that would place the enemy on a wall cell.